

Assignment No. 3

Identity and Access Management (IAM) — Practical Steps

Name	Sanket Kolhe	Roll No.	_____
Class / Division	_____	Subject	Cloud Computing (AWS)
Assignment No.	03	Date	05 / 09 / 2026

Q. Enlist the detailed steps involved in —

- a) Creation of IAM User and different ways of adding policies to it
- b) Creation of IAM Group and managing permission by adding user to the group
- c) Creation of IAM Role
- d) Assuming an IAM Role by an IAM User

Before We Start — IAM in Short

AWS Identity and Access Management (IAM) is the service that controls “who can do what” inside an AWS account. Every practical step below is done from the IAM console, so these terms must be clear first.

Term	Meaning	Key point
IAM User	A permanent identity for one person or one application.	Has permanent credentials — console password and/or access keys.
IAM Group	A collection of IAM users.	Only a container for permissions; it has no credentials and cannot log in.
IAM Role	An identity with permissions that is temporarily assumed.	No permanent credentials; gives temporary keys through AWS STS.
Policy	A JSON document that lists the permissions.	A policy does nothing until it is attached to a user, group or role.
Principal	The entity making the request (user, role, service).	Written in a policy as an ARN.
ARN	Amazon Resource Name — the unique address of a resource.	arn:aws:iam::123456789012:user/sanket-dev
AWS STS	Security Token Service.	Issues the temporary credentials when a role is assumed.

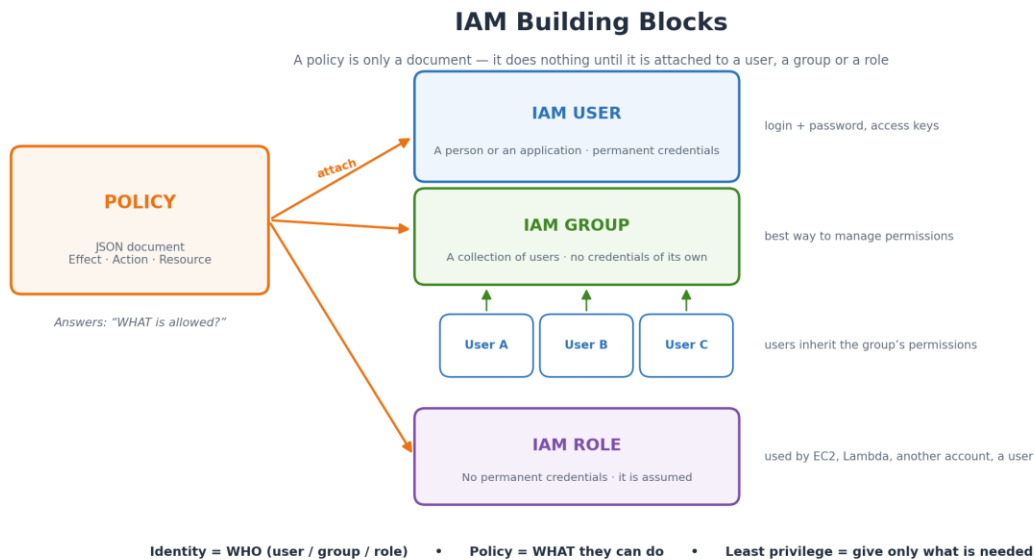


Figure 1: IAM building blocks — a policy is attached to a user, a group or a role.

Prerequisites

- **An AWS account** — sign in as the root user or as an IAM user that already has administrator permissions (AdministratorAccess).
- **Open the IAM console** — in the console search bar type “IAM” and open the service, or go to <https://console.aws.amazon.com/iam/>.
- **Remember: IAM is a global service** — there is no Region selector on IAM pages. A user or role created once works in every Region.

Note: For real human logins AWS now recommends AWS IAM Identity Center (the successor of AWS SSO) instead of individual IAM users. IAM users are still used for learning, for small accounts and for applications, which is why this practical uses them.

a) Creation of an IAM User

A.1 Steps to create the user (AWS Management Console)

- Step 1 Sign in** to the AWS Management Console as the root user or as an administrator IAM user.
- Step 2 Open the IAM console** — search “IAM” in the top search bar and click the IAM service.
- Step 3** In the left navigation pane click **Access management** → **Users**, then click the **Create user** button on the top right.
- Step 4 Specify user details.** Type the **User name** — for example **sanket-dev**. The name must be unique in the account.
- Step 5 Decide about console access.** Tick **“Provide user access to the AWS Management Console”** only if this user has to log in through the browser. Then choose:
 - **Console password** — Autogenerated password (AWS creates it) or Custom password (you type it).
 - **“Users must create a new password at next sign-in”** — keep this ticked so the user sets their own password on first login.

If the user is only for an application or the CLI, leave console access unticked — the user will use access keys instead.

Step 6 Click **Next** to reach the **Set permissions** page. Choose one of the three options (explained in detail in section A.3):

- **Add user to group** — recommended; the user inherits the group's policies.
- **Copy permissions** — copy everything from an existing user.
- **Attach policies directly** — pick individual policies for this one user.

Step 7 (Optional) Expand **Set permissions boundary** if the maximum permissions of this user must be limited. Leave it as “No permissions boundary” for a normal practical.

Step 8 Click **Next**. On the **Review and create** page check the user name and the permissions, and add **Tags** if needed (for example Key = Department, Value = IT).

Step 9 Click **Create user**.

Step 10 **Retrieve the password**. On the success page click **Download .csv file** or **Show** to copy the password, and note the **console sign-in URL**:

```
https://<your-12-digit-account-id>.signin.aws.amazon.com/console  
or the account alias URL:  
https://<account-alias>.signin.aws.amazon.com/console
```

Important: This is the only time the auto-generated password is shown. If it is lost, you must reset it from the user's Security credentials tab.

Step 11 **Verify**. Open a private/incognito window, go to the sign-in URL and log in with the new user to confirm it works.

A.2 Creating access keys (for CLI / SDK use)

Access keys are no longer created automatically with the user. Create them only if the user will use the AWS CLI, an SDK or an API.

Step 1 Open **IAM → Users** and click the user name.

Step 2 Go to the **Security credentials** tab, scroll to **Access keys** and click **Create access key**.

Step 3 Choose the use case — **Command Line Interface (CLI)** — tick the confirmation box and click **Next**.

Step 4 Add an optional description tag, then click **Create access key**.

Step 5 Click **Download .csv file**. The Secret access key is shown only once.

Step 6 Configure the CLI on your machine:

```
aws configure  
AWS Access Key ID      : AKIA.....  
AWS Secret Access Key  : *****  
Default region name    : ap-south-1  
Default output format  : json  
  
# check which identity the CLI is using  
aws sts get-caller-identity
```

Best practice: Never write access keys inside code or share them. For anything running on AWS (EC2, Lambda), do not create keys at all — attach an IAM role instead.

A.3 Different ways of adding policies to an IAM user

There are six practical ways to give permissions to a user. The first three are the ones used most often.

Way 1 — Attach an AWS managed policy directly

AWS managed policies are ready-made policies written and maintained by AWS (AmazonS3ReadOnlyAccess, AmazonEC2FullAccess, AdministratorAccess ...).

Step 1 Go to **IAM → Users → <user name> → Permissions** tab.

Step 2 Click **Add permissions → Add permissions**.

Step 3 Select **Attach policies directly**.

Step 4 In the search box type the policy name, for example **AmazonS3ReadOnlyAccess**, tick it, and click **Next → Add permissions**.

```
# CLI equivalent
aws iam attach-user-policy \
  --user-name sanket-dev \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

Way 2 — Create and attach a customer managed policy

Used when no AWS managed policy fits your exact requirement. It is reusable — the same policy can be attached to many users, groups and roles.

Step 1 Go to **IAM → Policies → Create policy**.

Step 2 Choose the **Visual** editor (pick Service, Actions, Resources) or the **JSON** tab and paste the document.

Step 3 Click **Next**, give a **Policy name** such as **S3-ReadOnly-MyBucket**, add a description and click **Create policy**.

Step 4 Now attach it: **Users → <user> → Add permissions → Attach policies directly**, filter by **Customer managed**, select it and add.

Example JSON policy — read-only access to one S3 bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListTheBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3::my-college-bucket"
    },
    {
      "Sid": "ReadTheObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3::my-college-bucket/*"
    }
  ]
}
```

Every policy has the same skeleton: Version, Statement, and inside each statement Effect (Allow/Deny), Action (the API calls), Resource (the ARN) and an optional Condition.

Way 3 — Through a group (recommended)

Attach the policies to a group once, then simply add users to that group. This is the method AWS recommends because permissions stay consistent and are easy to change.

Step 1 Create the group and attach policies to it (full steps in part b).

Step 2 Open IAM → Users → <user> → Groups tab → Add user to groups, select the group and confirm.

```
aws iam add-user-to-group --user-name sanket-dev --group-name Developers
```

Way 4 — Copy permissions from an existing user

Step 1 While creating a user (or from Add permissions), choose Copy permissions.

Step 2 Select the existing user. All its group memberships, attached managed policies and inline policies are copied to the new user.

Very useful when a new team member must get exactly the same access as an existing one.

Way 5 — Inline policy

An inline policy is embedded inside one user only. It has a strict one-to-one relationship: delete the user and the policy is deleted with it. Use it for a special, one-off permission.

Step 1 Go to IAM → Users → <user> → Permissions tab.

Step 2 Click Add permissions → Create inline policy.

Step 3 Write it in the Visual or JSON editor, click Next, give it a name and click Create policy.

```
aws iam put-user-policy --user-name sanket-dev \  
  --policy-name DenyDeleteBucket \  
  --policy-document file://policy.json
```

Way 6 — Permissions boundary (advanced)

A permissions boundary does not grant anything. It sets the maximum permissions a user can ever have. The effective permission is the overlap of the boundary and the attached policies.

Step 1 Go to IAM → Users → <user> → Permissions tab → Permissions boundary → Set boundary.

Step 2 Select a managed policy to act as the boundary and save.

Example: If the boundary allows only S3 and the attached policy allows S3 + EC2, the user effectively gets only S3.
Boundary $\cap$ Policy = final permission.

Comparison of the main ways:

Way	Reusable?	When to use	Deleted with the user?
AWS managed policy	Yes — AWS maintains it	Standard, common permissions	No
Customer managed policy	Yes — you maintain it	Your own exact requirement	No
Group membership	Yes	Teams / many users — best practice	No
Copy permissions	One-time copy	New member of an existing team	No
Inline policy	No — one user only	A special one-off permission	Yes

Permissions boundary	Yes	To cap the maximum permissions	No
----------------------	-----	--------------------------------	----

b) Creation of an IAM Group and Managing Permissions

A group is a container of users. Policies attached to the group are automatically inherited by every user inside it, so permissions for a whole team can be changed in one place.

B.1 Create the group

Step 1 Open the **IAM console**.

Step 2 In the left pane click **Access management** → **User groups**, then **Create group**.

Step 3 Enter the **User group name** — for example **Developers**. Use a name that describes the job role.

Step 4 (Optional) In **Add users to the group**, tick the users who should be members right now.

Step 5 In **Attach permissions policies**, search and tick the policies the whole team needs, for example **AmazonS3ReadOnlyAccess** and **AmazonEC2ReadOnlyAccess**.

Step 6 Click **Create user group**.

```
# CLI
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
    --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

B.2 Add a user to the group (two ways)

From the group side:

Step 1 Go to **IAM** → **User groups** → <group name>.

Step 2 Open the **Users** tab and click **Add users**.

Step 3 Tick the users and click **Add users**.

From the user side:

Step 1 Go to **IAM** → **Users** → <user name>.

Step 2 Open the **Groups** tab and click **Add user to groups**.

Step 3 Select the group and confirm.

```
aws iam add-user-to-group --user-name sanket-dev --group-name Developers
aws iam get-group --group-name Developers          # list the members
```

B.3 Manage the group's permissions

Step 1 Open **IAM** → **User groups** → <group> → **Permissions** tab.

Step 2 To add: **Add permissions** → **Attach policies** → select → **Attach policies**.

Step 3 To remove: click **Remove** next to the policy (or **Detach**).

Step 4 To remove a user from the group: **Users** tab → tick the user → **Remove**.

Any change here applies immediately to every user in the group.

```
aws iam remove-user-from-group --user-name sanket-dev --group-name Developers
aws iam detach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

B.4 Verify

- Log in as the user and open the S3 console — the buckets should be visible (read-only).
- Try to delete an object — it must fail with an “access denied” message, which proves the group policy is working.
- Or use IAM → Users → <user> → Access Advisor to see which services the user has permission for and last used.

Important points about groups: A group is not an identity — it cannot be named as a Principal in a policy and it has no credentials. Groups cannot be nested (a group cannot contain another group). One user can belong to up to 10 groups by default.

c) Creation of an IAM Role

A role is an identity that has permission policies but no permanent credentials. Instead of logging in, an entity “assumes” the role and receives temporary credentials from AWS STS. Roles are the safe way to give access to EC2 instances, Lambda functions, other AWS accounts and federated users.

Every role always has two policies:

Policy	Question it answers	Example
Trust policy	WHO is allowed to assume this role?	The EC2 service, another AWS account, an IAM user.
Permissions policy	WHAT can the role do after it is assumed?	AmazonS3FullAccess, a custom policy, etc.

C.1 Trusted entity types offered by the console

Trusted entity	Used for
AWS service	EC2, Lambda, ECS ... — the most common; lets a service act on your behalf.
AWS account	Cross-account access, or letting an IAM user of this account assume the role.
Web identity	Login through Google, Facebook, Amazon or another OIDC provider.
SAML 2.0 federation	Company directory login such as Active Directory.
Custom trust policy	You write the trust JSON yourself for full control.

C.2 Steps to create a service role (example: EC2 needs to read S3)

Step 1 Open IAM → Access management → Roles and click **Create role**.

Step 2 For **Trusted entity type** select **AWS service**.

Step 3 Under **Use case** choose **EC2** and click **Next**.

Step 4 On **Add permissions**, search and tick **AmazonS3ReadOnlyAccess** (or your own policy). Click **Next**.

Step 5 Enter a **Role name** such as **EC2-S3-ReadOnly-Role** and a description.

Step 6 Check the **Trust policy** shown on the review page — it should name the EC2 service:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

Step 7 Click **Create role**.

Step 8 Attach the role to an EC2 instance: go to the **EC2 console** → **Instances** → select the instance → **Actions** → **Security** → **Modify IAM role** → choose the role → **Update IAM role**.

Step 9 Verify — connect to the instance and run:

```
aws sts get-caller-identity    # shows the assumed-role ARN
aws s3 ls                    # works without any access keys on the instance
```

Why this matters: The instance now works without any access keys stored on the disk. AWS rotates the temporary credentials automatically. This is the single most important reason roles exist.

C.3 Steps to create a cross-account role

Step 1 In the **trusting account** (the one that owns the resources) go to **Roles** → **Create role**.

Step 2 Select **AWS account** → **Another AWS account** and type the 12-digit **Account ID** of the trusted account.

Step 3 (Recommended) Tick **Require external ID** for third-party access, and **Require MFA** for extra security.

Step 4 Attach the permissions policy, name the role, and create it.

Step 5 In the **trusted account**, give the user permission to call **sts:AssumeRole** on that role ARN (see part d).

d) Assuming an IAM Role by an IAM User

“Assuming a role” means an IAM user temporarily gives up its own permissions and works with the role's permissions instead. AWS STS issues short-lived credentials for the session.

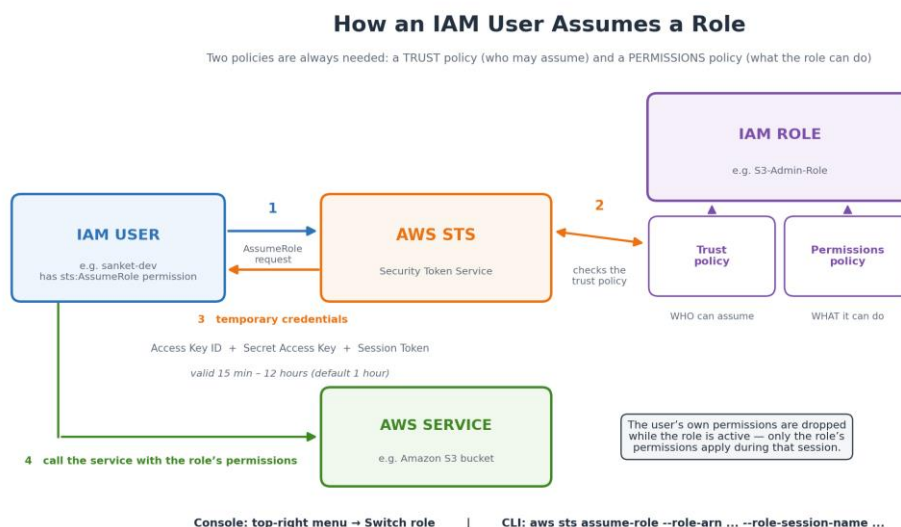


Figure 2: The assume-role flow — the user calls STS, STS checks the trust policy and returns temporary credentials.

Two permissions must exist — this is the point students most often miss:

Side	What is needed	Where it is written
On the role	The trust policy must name the user (or the account) as a principal.	Role → Trust relationships tab
On the user	An identity policy allowing sts:AssumeRole on that role ARN.	User → Permissions tab

D.1 Create the role that will be assumed

Step 1 Go to **IAM → Roles → Create role**.

Step 2 Choose **Custom trust policy** (or AWS account → This account) and paste the trust policy naming your user:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::123456789012:user/sanket-dev"
    },
    "Action": "sts:AssumeRole"
  }]
}
```

Step 3 Click **Next** and attach the permissions policy the role should have, for example **AmazonS3FullAccess**.

Step 4 Name the role **S3-Admin-Role** and click **Create role**.

Step 5 Open the role and copy its **Role ARN** — you will need it:

```
arn:aws:iam::123456789012:role/S3-Admin-Role
```

D.2 Allow the user to assume the role

Step 1 Go to **IAM → Users → sanket-dev → Permissions tab → Add permissions → Create inline policy → JSON tab**.

Step 2 Paste this policy and save it as **AllowAssumeS3AdminRole**:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "sts:AssumeRole",
    "Resource": "arn:aws:iam::123456789012:role/S3-Admin-Role"
  }]
}
```

D.3 Switch to the role in the console

Step 1 Sign in to the console as the IAM user **sanket-dev**.

Step 2 Click the **account menu at the top right** and choose **Switch role**.

Step 3 Click **Switch role** on the information page and fill in:

- **Account** — the 12-digit account ID (or the account alias).

- **Role** — the exact role name, S3-Admin-Role.
- **Display Name** — any label you like, e.g. S3 Admin.
- **Colour** — pick a colour so you can see at a glance that you are in the role.

Step 4 Click **Switch Role**. The top-right corner now shows the display name and colour.

Step 5 Test it — open S3 and create or delete an object. It now works, even though the user itself has no S3 permission.

Step 6 To go back, open the same menu and click **Back to <user name>**.

The console remembers the last 5 roles you switched to, so next time you can pick from the list. Note that the root user cannot switch roles.

D.4 Assume the role from the AWS CLI

Method 1 — call STS directly:

```
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/S3-Admin-Role \
  --role-session-name sanket-session-1

# the reply contains three values:
#  AccessKeyId, SecretAccessKey, SessionToken

# use them for the next commands (Linux / Git Bash):
export AWS_ACCESS_KEY_ID=ASIA.....
export AWS_SECRET_ACCESS_KEY=*****
export AWS_SESSION_TOKEN=IQoJb3JpZ2luX2VjE...

aws sts get-caller-identity      # should now show assumed-role/S3-Admin-Role
aws s3 ls
```

Method 2 — a named profile (cleaner, the CLI assumes the role for you):

```
# file: ~/.aws/config      (Windows: C:\Users\<<name>\.aws\config)
[profile s3admin]
role_arn      = arn:aws:iam::123456789012:role/S3-Admin-Role
source_profile = default
region        = ap-south-1

# then simply:
aws s3 ls --profile s3admin
```

The temporary credentials are valid for 1 hour by default. The maximum session duration can be set between 1 and 12 hours on the role's Summary page.

D.5 (Optional) Force MFA while assuming the role

Add a Condition to the trust policy so the role can be assumed only if the user has signed in with MFA:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::123456789012:user/sanket-dev" },
    "Action": "sts:AssumeRole",
    "Condition": {
      "Bool": { "aws:MultiFactorAuthPresent": "true" }
    }
  }]
}
```

Important Points to Remember

1. How AWS decides Allow or Deny

- **Default is deny** — if nothing allows the action, it is denied (implicit deny).
- **An explicit Allow** in any attached policy permits the action.
- **An explicit Deny always wins** — even if ten other policies allow it. This is the most important rule of policy evaluation.
- **Permissions add up** — a user gets the union of its own policies plus every group's policies, then capped by any permissions boundary or SCP.

2. User vs Group vs Role — quick comparison

Point	IAM User	IAM Group	IAM Role
Credentials	Permanent (password / access keys)	None	Temporary, from STS
Can log in?	Yes	No	Only by assuming it
Used by	A person or an application	A set of users	AWS services, users, other accounts
Policy attached?	Yes	Yes	Yes + a trust policy
Best for	One identity	Managing a team's permissions	Temporary or machine access

3. Managed policy vs Inline policy

Point	Managed policy	Inline policy
Reusable	Yes — attach to many identities	No — belongs to one identity only
Types	AWS managed and customer managed	Only yours
Versioning	Customer managed policies keep up to 5 versions	No versioning
If identity deleted	Policy stays	Policy is deleted too
Recommended	Yes, for almost everything	Only for a strict one-to-one exception

4. Security best practices

- **Least privilege** — start with the minimum permissions and add more only when needed.
- **Do not use the root account for daily work** — enable MFA on it and lock it away.
- **Enable MFA for all users, especially administrators.**
- **Prefer roles over access keys** — never store keys on an EC2 instance or in code.
- **Rotate access keys regularly** and delete the ones that are unused.
- **Use groups, not individual attachments** — it keeps permissions consistent.
- **Review the Credential Report and Access Advisor** — IAM → Credential report shows every user's keys and MFA status; Access Advisor shows which services a user actually used.
- **Use IAM Access Analyzer** to find resources shared outside the account.

5. Useful facts and default limits

- **IAM is a global service** — no Region selection; changes apply everywhere.
- **IAM is free** — users, groups, roles and policies cost nothing.
- **Default quotas** — 5,000 users, 300 groups and 1,000 roles per account; a user can be in 10 groups; 10 managed policies can be attached to a user, group or role.
- **Temporary credentials** — 15 minutes to 12 hours; the default is 1 hour.
- **A user name can be changed later**, but the ARN changes with it.
- **Policy changes take effect immediately** — no restart is needed.

6. Common errors and how to fix them

Error message	Reason	Fix
User is not authorized to perform sts:AssumeRole	The user has no policy allowing sts:AssumeRole.	Add the inline policy from step D.2.
Invalid information in one or more fields (on Switch role)	Wrong account ID or role name, or the trust policy does not include the user.	Check the role ARN and the Trust relationships tab.
AccessDenied on an S3 / EC2 action	The permissions policy of the role or user does not allow that action.	Attach the correct policy; check for an explicit Deny.
The security token included in the request is expired	The temporary credentials have crossed their session duration.	Run assume-role again to get fresh credentials.
Access keys not shown after creating a user	AWS no longer creates keys automatically.	Create them from the Security credentials tab.

7. Viva / interview questions on this practical

Q1. Why use a role instead of access keys on an EC2 instance?

Ans. Roles give temporary credentials that AWS rotates automatically, so no long-term secret is stored on the instance where it could be stolen.

Q2. What are the two policies attached to every role?

Ans. A trust policy (who may assume it) and a permissions policy (what it may do).

Q3. Can a group be a principal in a policy?

Ans. No. A group is only a container of users; it is not an identity and has no ARN that can be used as a principal.

Q4. What happens to the user's own permissions while a role is assumed?

Ans. They are not used. Only the role's permissions apply for that session.

Q5. Which wins — an Allow or a Deny?

Ans. An explicit Deny always wins over any Allow.

Q6. Difference between a permissions boundary and a policy?

Ans. A policy grants permissions; a boundary only sets the maximum limit and grants nothing.

Q7. Is IAM regional or global?

Ans. Global — an IAM user or role works in every Region.

Q8. Which service issues the temporary credentials?

Ans. AWS STS (Security Token Service).

Conclusion

In this practical we created an IAM user and saw six different ways of giving it permissions, created a group and managed a whole team's access from one place, created a role with its trust and permissions policies, and finally assumed that role from an IAM user through both the console and the CLI. The key learning is that IAM separates **identity** (who — user, group, role) from **permission** (what — the policy), and that roles with temporary credentials are always safer than permanent access keys.